K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Database Management System - ENCS254

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

2nd APRIL, 2026

(CAPSTONE PROJECT)

MUSIC LIBRARY MANAGEMENT SYSTEM

SUBMITTED BY

SNEHA GUPTA
2401010040

SUBMITTED TO

Ms. MANSI KAJAL

1. INTRODUCTION

The Music Library Management System is designed to manage and organize music-related data such as songs, artists, albums, and genres. It helps in storing, retrieving, and analyzing music information efficiently using a relational database. The system also allows users to search songs, manage playlists, and perform various database operations.

2. OBJECTIVES

- To design a database schema for music library
- To manage songs, artists, albums, and genres
- To perform CRUD operations
- To execute SQL queries for data retrieval and analysis

3. ER DIAGRAM

Entities:

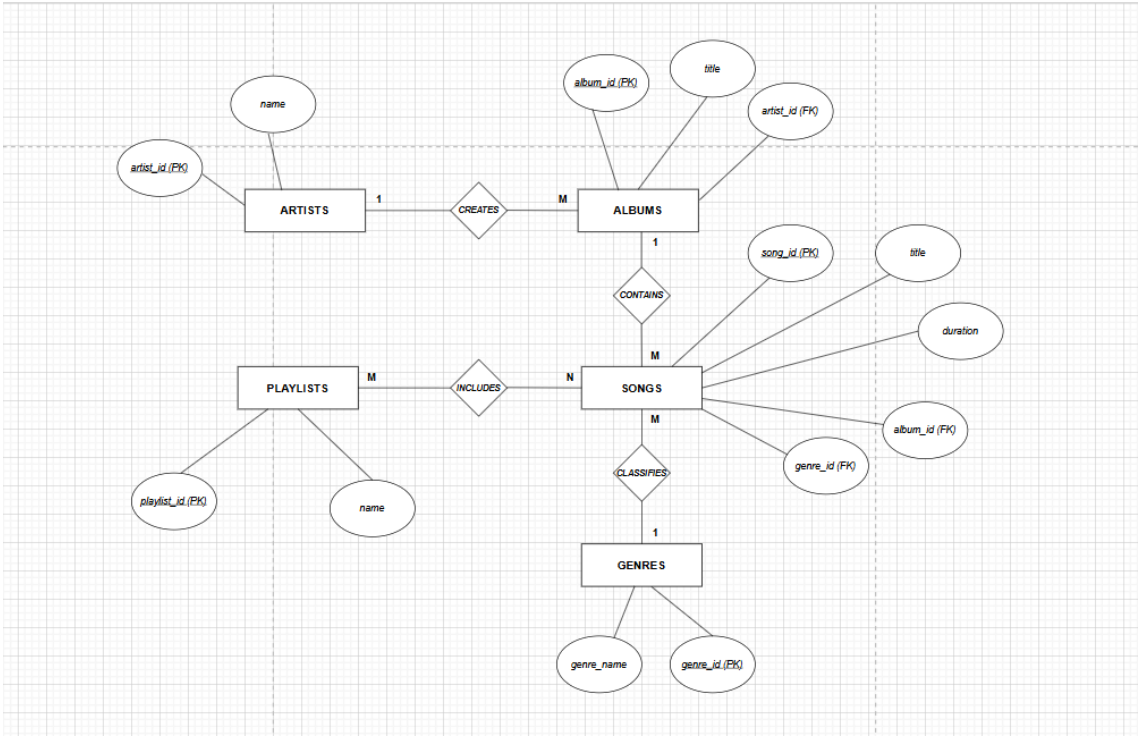
1. Artists
2. Albums
3. Songs
4. Genres
5. Playlists

Attributes:

Artists - artist_id (PK) , name
Albums - album_id (PK) , title , artist_id(FK)
Songs - song_id(PK) , title , duration , album_id (FK) , genre_id(FK)
Genres - genre_id(PK), genre_name
Playlists - playlist_id(PK) , name

Relationships:

Artist → Album (1:M) ---- Creates
Album → Song (1:M) --- Contains
Genre → Song (1:M) --- Classifies
Playlist ↔ Song (M:N) --- Includes



4. DATABASE SCHEMA

Description:
The database consists of multiple tables to store music-related data. Relationships are maintained using foreign keys.

SQL TABLE CREATION:

```
mysql> create database capstone;
Query OK, 1 row affected (0.08 sec)

mysql> use capstone;
Database changed
mysql> CREATE TABLE Artists (
    ->     artist_id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql>
mysql> CREATE TABLE Genres (
    ->     genre_id INT PRIMARY KEY,
    ->     genre_name VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Albums (
    ->     album_id INT PRIMARY KEY,
    ->     title VARCHAR(100),
    ->     artist_id INT,
    ->     FOREIGN KEY (artist_id) REFERENCES Artists(artist_id)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> CREATE TABLE Songs (
    ->     song_id INT PRIMARY KEY,
    ->     title VARCHAR(100),
    ->     duration INT,
    ->     album_id INT,
    ->     genre_id INT,
    ->     FOREIGN KEY (album_id) REFERENCES Albums(album_id),
    ->     FOREIGN KEY (genre_id) REFERENCES Genres(genre_id)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> CREATE TABLE Playlists (
    ->     playlist_id INT PRIMARY KEY,
    ->     name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Playlist_Songs (
    ->     playlist_id INT,
    ->     song_id INT,
    ->     PRIMARY KEY (playlist_id, song_id),
    ->     FOREIGN KEY (playlist_id) REFERENCES Playlists(playlist_id),
    ->     FOREIGN KEY (song_id) REFERENCES Songs(song_id)
    -> );
```

5. INSERT SAMPLE DATA

Description:
Sample data is inserted to test the functionality of the database.

```
mysql> INSERT INTO Artists VALUES (1, 'Arijit Singh'), (2, 'Taylor Swift');
Query OK, 2 rows affected (0.12 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Genres VALUES (1, 'Romantic'), (2, 'Pop');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Albums VALUES (1, 'Album1', 1), (2, 'Album2', 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Songs VALUES
    -> (1, 'SongA', 210, 1, 1),
    -> (2, 'SongB', 200, 2, 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Playlists VALUES (1, 'MyPlaylist');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO Playlist_Songs VALUES (1, 1), (1, 2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

6. SQL QUERIES WITH OUTPUT

Query 1: Display all songs

```
mysql> SELECT * FROM Songs;
+---------+-------+----------+----------+----------+
| song_id | title | duration | album_id | genre_id |
+---------+-------+----------+----------+----------+
|       1 | SongA |      210 |        1 |        1 |
|       2 | SongB |      200 |        2 |        2 |
+---------+-------+----------+----------+----------+
2 rows in set (0.02 sec)
```

Query 2: Display songs with artist name

```
mysql> SELECT s.title, a.name
    -> FROM Songs s
    -> JOIN Albums al ON s.album_id = al.album_id
    -> JOIN Artists a ON al.artist_id = a.artist_id;
+-------+--------------+
| title | name         |
+-------+--------------+
| SongA | Arijit Singh |
| SongB | Taylor Swift |
+-------+--------------+
2 rows in set (0.04 sec)
```

Query 3: Display songs by genre

```
mysql> SELECT s.title, g.genre_name
    -> FROM Songs s
    -> JOIN Genres g ON s.genre_id = g.genre_id;
+-------+------------+
| title | genre_name |
+-------+------------+
| SongA | Romantic   |
| SongB | Pop        |
+-------+------------+
2 rows in set (0.00 sec)
```

Query 4: Count songs per artist

```
mysql> SELECT a.name, COUNT(s.song_id) AS total_songs
    -> FROM Artists a
    -> JOIN Albums al ON a.artist_id = al.artist_id
    -> JOIN Songs s ON al.album_id = s.album_id
    -> GROUP BY a.name;
+--------------+-------------+
| name         | total_songs |
+--------------+-------------+
| Arijit Singh |           1 |
| Taylor Swift |           1 |
+--------------+-------------+
2 rows in set (0.02 sec)
```

Query 5: Display playlist songs

```
mysql> SELECT p.name, s.title
    -> FROM Playlists p
    -> JOIN Playlist_Songs ps ON p.playlist_id = ps.playlist_id
    -> JOIN Songs s ON ps.song_id = s.song_id;
+------------+-------+
| name       | title |
+------------+-------+
| MyPlaylist | SongA |
| MyPlaylist | SongB |
+------------+-------+
2 rows in set (0.01 sec)
```

Query 6: Update song name

```
mysql> UPDATE Songs
    -> SET title = 'Updated Song'
    -> WHERE song_id = 1;
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Query 7: Delete a song

```
mysql> DELETE FROM Songs WHERE song_id = 2;
Query OK, 1 row affected (0.01 sec)
```

Query 8: Songs longer than 3 minutes

```
mysql> SELECT * FROM Songs WHERE duration > 180;
+---------+--------------+----------+----------+----------+
| song_id | title        | duration | album_id | genre_id |
+---------+--------------+----------+----------+----------+
|       1 | Updated Song |      210 |        1 |        1 |
+---------+--------------+----------+----------+----------+
1 row in set (0.00 sec)
```

7. CRUD OPERATIONS

Description:

- **Create:** Insert new records
- **Read:** Retrieve data using SELECT
- **Update:** Modify existing records
- **Delete:** Remove records

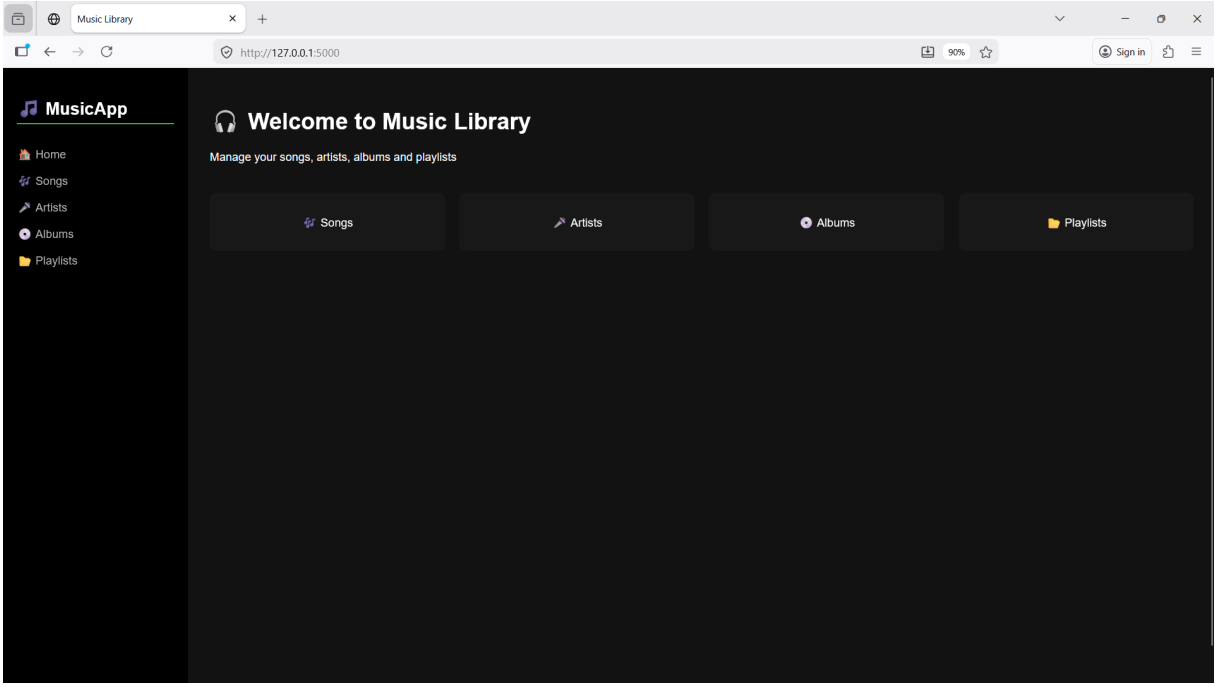
(Already done in above sql queries!!!)

8. CONSTRAINTS USED

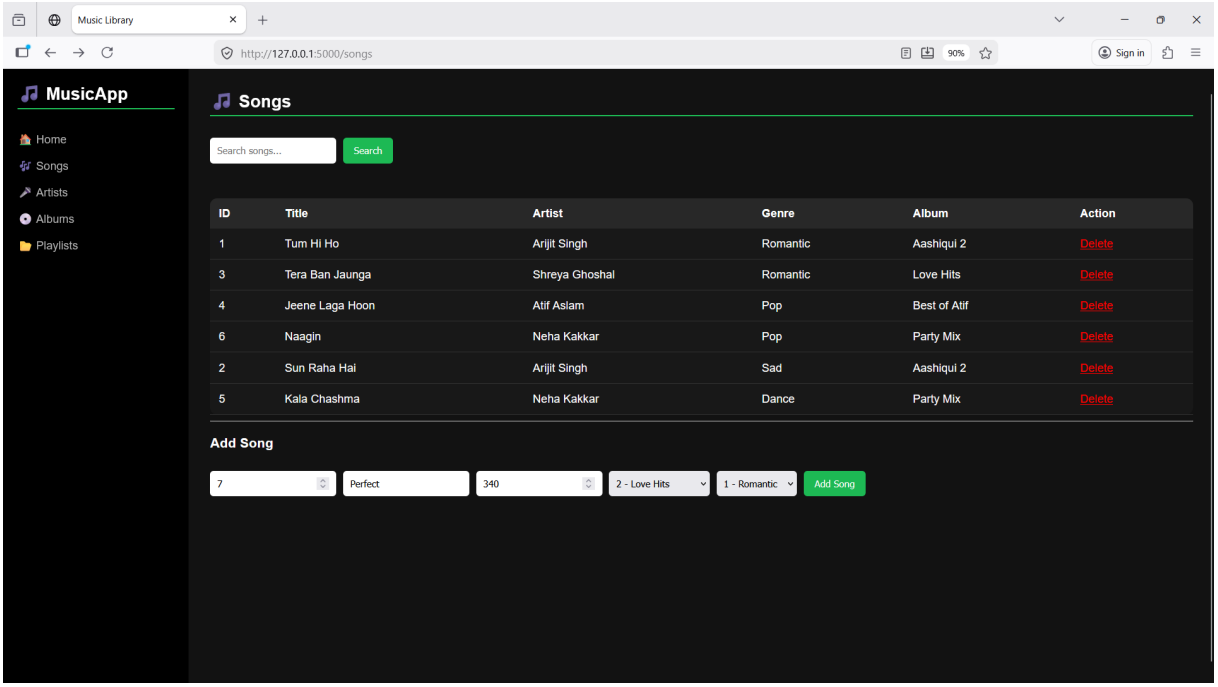
- **Primary Key:** Uniquely identifies each record
- **Foreign Key:** Maintains relationships between tables
- **NOT NULL:** Prevents empty values

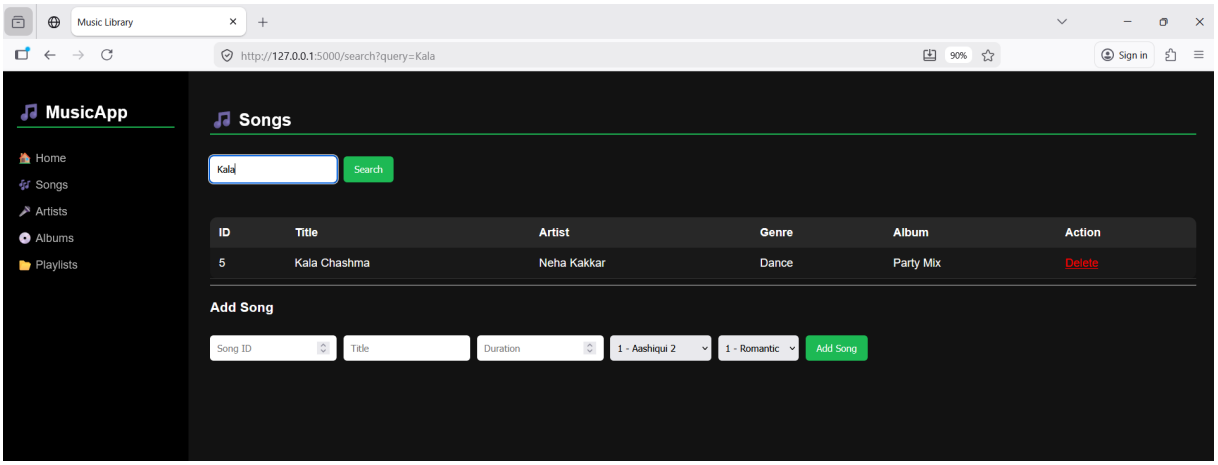
9. UI + DB (Website)

Landing Page:

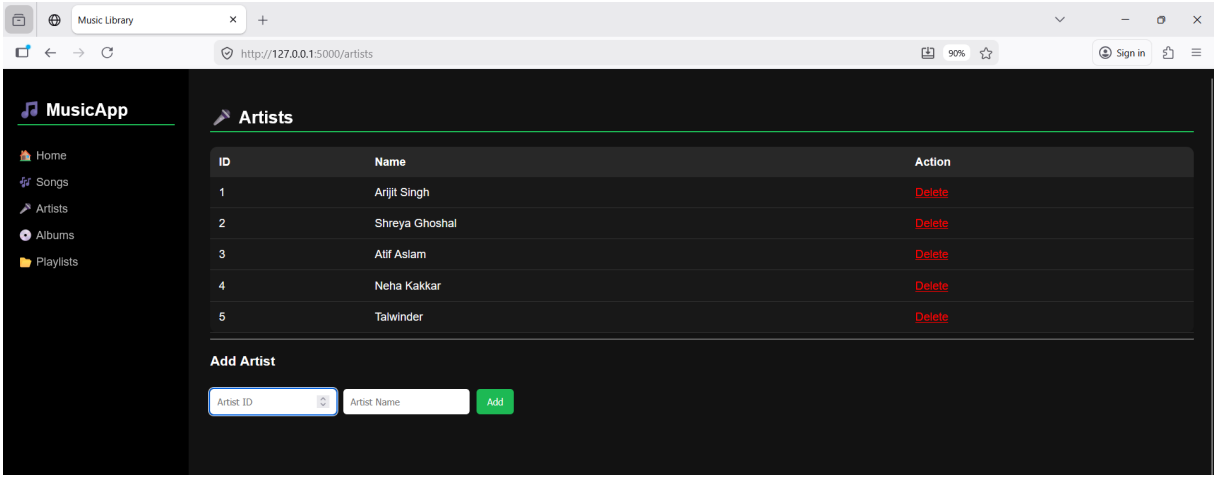


Songs:

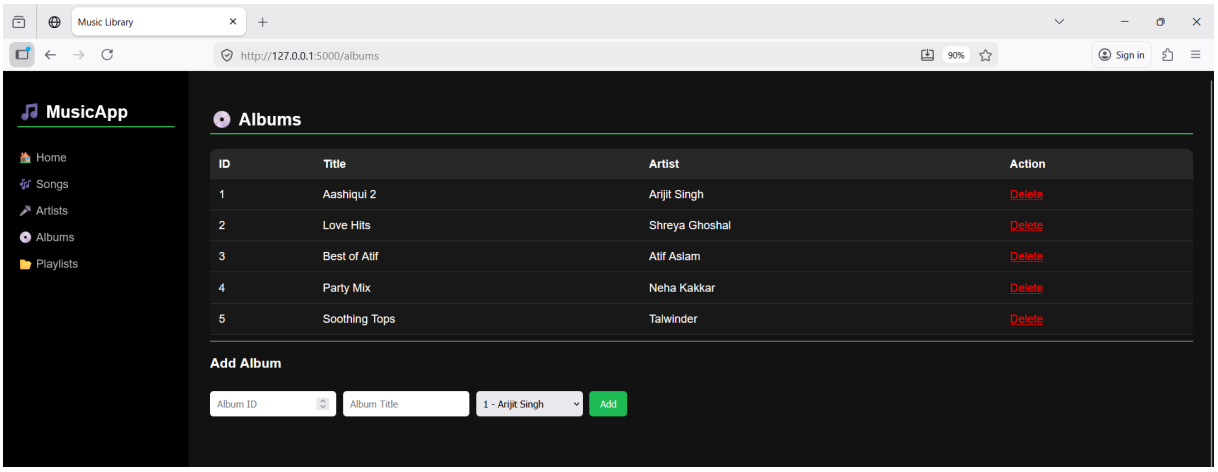

Music Library
http://127.0.0.1:5000/songs
90%
Sign in
MusicApp
Home
Songs
Artists
Albums
Playlists
Songs
Search songs...
Search
ID Title Artist Genre Album Action
1 Tum Hi Ho Arijit Singh Romantic Aashiqui 2 Delete
3 Tera Ban Jaunga Shreya Ghoshal Romantic Love Hits Delete
4 Jeene Laga Hoon Atif Aslam Pop Best of Atif Delete
6 Naagin Neha Kakkar Pop Party Mix Delete
2 Sun Raha Hai Arijit Singh Sad Aashiqui 2 Delete
5 Kala Chashma Neha Kakkar Dance Party Mix Delete
Add Song
7
Perfect
340
2 - Love Hits
1 - Romantic
Add Song


Music Library
http://127.0.0.1:5000/search?query=Kala
90%
Sign in
MusicApp
Home
Songs
Artists
Albums
Playlists
Songs
Kala
Search
ID Title Artist Genre Album Action
5 Kala Chashma Neha Kakkar Dance Party Mix Delete
Add Song
Song ID
Title
Duration
1 - Aashiqui 2
1 - Romantic
Add Song

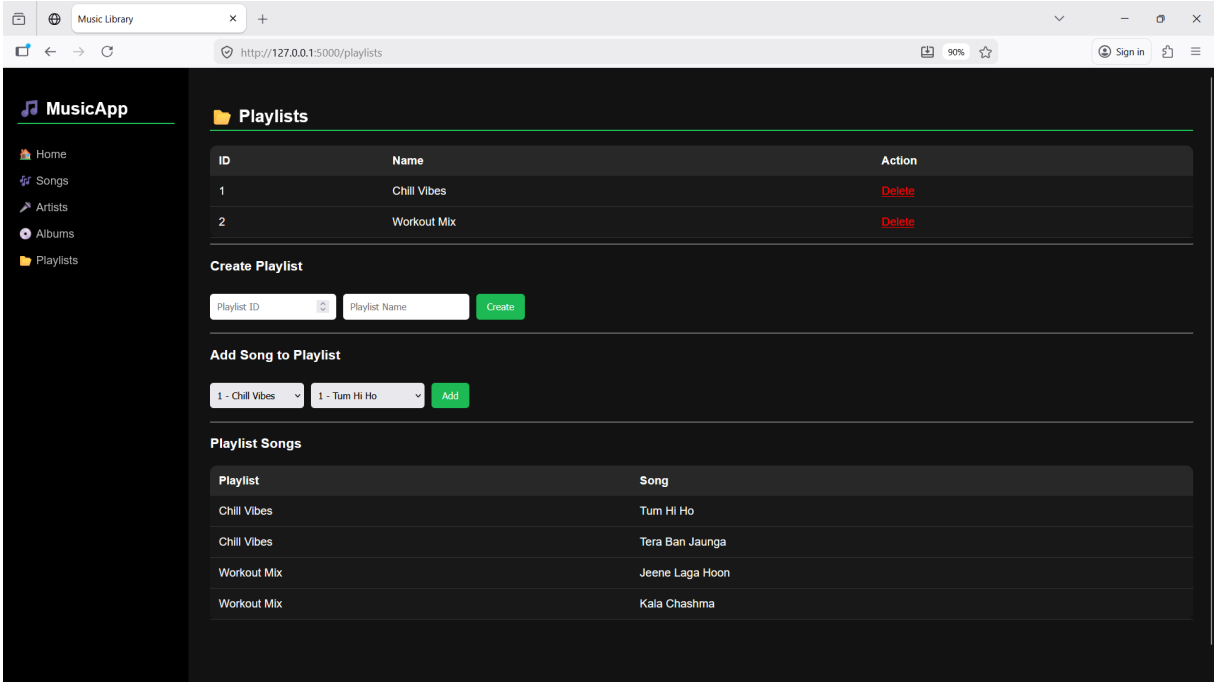
Artists:



Albums:



Playlists:



10. CONCLUSION

The Music Library Management System successfully demonstrates database design and implementation using SQL. It allows efficient storage and retrieval of music data. The system can be further enhanced with additional features such as user authentication and recommendation systems.